Information Exposure Maximization in Social Networks: An Empirical Study of Monte Carlo-Guided Heuristics and Evolutionary Algorithms

Yanqiao Chen
Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

Abstract

Information Exposure Maximization (IEM) aims to balance the spread of competing campaigns and mitigate information disparity in social networks. Driven by the NP-hard nature of the diffusion models, we empirically investigate two distinct approaches relying on Monte Carlo (MC) sampling: an MC-SAA Greedy Heuristic with IMRank candidate screening, and a Binary Evolutionary Algorithm with Simulated Annealing refinement. Rather than performing a strict performance comparison between the two—given their differing scalability constraints and dataset configurations—this study focuses on characterizing their individual behaviors under various graph topologies and propagation probabilities. Furthermore, we conduct a cross-architecture hardware benchmark to reveal how algorithmic bottlenecks respond to modern CPU upgrades. Our findings underscore that computational efficiency is dominated not merely by graph size, but by the effective diffusion range dictated by transmission probabilities.

I Introduction

Information Exposure Maximization (IEM) studies how to reduce information disparity between two competing campaigns over a directed social network. Given initial seed sets I_1 and I_2, we select additional seed sets S_1 and S_2 under a total budget constraint to maximize balanced exposure under Independent Cascade (IC) propagation.

Let R_1 and R_2 denote the reached node sets of campaign 1 and campaign 2 in one diffusion realization. The optimization target is

$$\Phi = |V - (R_1 \oplus R_2)| = |V| - |R_1 \oplus R_2|, \tag{1}$$

where $\oplus$ is symmetric difference. The expected objective is

$$\max_{S_1,S_2} \mathbb{E}[\Phi], \quad \text{s.t. } |S_1| + |S_2| \le k,\ S_1 \cap S_2 = \varnothing, \tag{2}$$

with full seeds $I_1 \cup S_1$ and $I_2 \cup S_2$.

This report implements and compares three core algorithmic components:

- Monte Carlo IC Evaluator for unbiased objective estimation.
- MC-SAA Greedy Heuristic with IMRank candidate screening.
- Binary Evolutionary Optimization using pymoo with feasibility repair.

We implement experiments on provided and generated datasets, with detailed timing and score reporting for both heuristic and evolutionary pipelines. The report also includes a cross-machine performance analysis to understand hardware impacts on runtime. All algorithms and experiments are designed for reproducibility, with fixed random seeds and batch scripts for execution. See Github for complete scripts and data: https://github.com/hotteano/Artificial-Intelligence-H-Course-Material-and-Notes-for-SPR26-in-SUSTech.

I-A Related Work

Information Exposure Maximization (IEM) was first formalized by Garimella et al. [1], who studied how to reduce information disparity between opposing campaigns in social networks. Their work established the IC-based exposure balancing framework and showed that even simple greedy heuristics can be effective for this objective. However, they focused on unbiased Monte Carlo evaluation without exploring algorithmic optimizations for large-scale graphs.

For single-campaign Influence Maximization (IM), extensive research has developed efficient approximation algorithms. IMRank [2] provides a fast ranking-based method for candidate screening, which we adapt for the IEM context to reduce marginal gain evaluation costs. While IMRank was originally designed for single-campaign influence spread, our work demonstrates its effectiveness for dual-campaign exposure balancing.

Simulated Annealing (SA) has been explored for influence maximization problems. Wu et al. [3] proposed a boost SA approach for budgeted influence maximization, showing that SA can escape local optima better than greedy methods when the objective landscape is rugged. Our work extends this insight to the IEM setting, combining GA global exploration with SA local refinement, and introducing an adaptive mechanism to disable SA for dense graphs where its overhead outweighs benefits.

I-B Terminology

- **Heuristic** refers to the MC-SAA Greedy method with IMRank screening.
- **Evolutionary** refers to the Binary GA + SA Refinement method.
- **Evaluator** refers to the Monte Carlo IC Evaluator used for final score estimation.
- **IEM** stands for Information Exposure Maximization.
- **GA** stands for Genetic Algorithm.
- **SA** stands for Simulated Annealing.
- **MC** stands for Monte Carlo.

II Method

In this section, we introduce the three core algorithms in our IEM pipeline: the Monte Carlo IC Evaluator, the MC-SAA Greedy Heuristic with IMRank Screening, and the Binary Evolutionary Optimization with Simulated Annealing Refinement. Each algorithm is designed to balance solution quality and computational efficiency under the constraints of the IEM problem.

II-A Algorithm A: Monte Carlo IC Evaluator

Computing the IEM objective Φ exactly is #P-hard due to the combinatorial nature of influence spread under the IC model. Therefore, we use a Monte Carlo (MC) simulation approach to estimate $\mathbb{E}[\Phi]$ for given seed sets S_1 and S_2.

For each simulation, both campaigns diffuse independently under IC. A node is considered reached if it is activated or attempted. The score of one simulation is $|V| - |R_1 \oplus R_2|$, and the final estimate is the sample mean over T runs:

$$\hat{\Phi}(S_1, S_2) = \frac{1}{T} \sum_{t=1}^{T} \left(|V| - |R_1^{(t)} \oplus R_2^{(t)}| \right). \tag{3}$$

See 1 for the detailed algorithm. The computational complexity of this evaluator is $O(T \cdot \text{diffusion_cost})$, where diffusion_cost depends on the graph structure and propagation probabilities. In practice, we find that $T = 5000$ provides a good balance between estimation accuracy and runtime for our test cases.

Algorithm 1 Monte Carlo IC Evaluation

Require: Graph G, probabilities (p_1, p_2), initial seeds (I_1, I_2), additional seeds (S_1, S_2), simulations T

```
sum ← 0
for t = 1 to T do
    Simulate IC for campaign 1 from I_1 ∪ S_1, get R_1^(t)
    Simulate IC for campaign 2 from I_2 ∪ S_2, get R_2^(t)
    sum ← sum + (|V| − |R_1^(t) ⊕ R_2^(t)|)
end for
return sum/T
```

II-B Algorithm B: MC-SAA Greedy with IMRank Screening

The heuristic iteratively allocates one budget unit to either campaign 1 or campaign 2. In each step, it samples N Monte Carlo base scenarios from current seeds. For each candidate node v, it estimates two marginal gains by incremental diffusion:

$$h_1(v) = \mathbb{E}\left[2\,|\Delta R_1(v) \cap R_2| - |\Delta R_1(v)|\right], \tag{4}$$

$$h_2(v) = \mathbb{E}\left[2\,|\Delta R_2(v) \cap R_1| - |\Delta R_2(v)|\right]. \tag{5}$$

Here $\Delta R_i(v)$ is the newly reached set when adding v to campaign i on top of a sampled base state. The better of $\max_v h_1(v)$ and $\max_v h_2(v)$ is selected.

To reduce candidate evaluation cost, IMRank [2] provides a ranked pool from both campaigns, then only top candidates are evaluated in each greedy step. Under graphs with higher probabilities, each Monte Carlo simulation tends to reach more nodes, which can make incremental diffusion more expensive. IMRanking helps focus on promising candidates, thus controlling runtime while maintaining solution quality. See 2 for the detailed algorithm.

The computational complexity of this heuristic is $O(k \cdot C \cdot N \cdot \text{diffusion_cost})$, where C is the candidate pool size and diffusion_cost depends on graph structure and probabilities. To be specific, diffusion cost of one incremental evaluation is $O(|E|)$ in worst case, but often much less in practice due to early termination when no new nodes are reached. In graph with higher transition probability, the cost will get higher. We test this property by experiment in the heuristic part, map5, which is a generated case designed to have higher probability and community structure. By tuning C and N, we can balance runtime and solution quality effectively.

II-C Algorithm C: Binary Evolutionary Optimization with Simulated Annealing Refinement

The evolutionary solver encodes solution as a binary chromosome

$$x = [x^{(1)} \,|\, x^{(2)}] \in \{0, 1\}^{2n_a}, \tag{6}$$

where n_a is the number of available nodes, and bits in each half indicate inclusion in S_1 or S_2. Fitness is coarse Monte Carlo score with a tiny budget-usage tie-break bonus; pymoo minimizes $F(x) = -\text{fitness}(x)$.

Feasibility is enforced by repair:

- remove dual assignments ($x_i^{(1)} = x_i^{(2)} = 1$),
- if selected count exceeds budget, randomly drop extras,
- if below budget, randomly fill free nodes to reach target.

Simulated Annealing Refinement. After the GA converges, a simulated annealing (SA) stage refines the best chromosome locally. SA performs M steps starting from x^*; at each step it generates a neighbor by either (1) swapping a selected node between campaigns, or (2) replacing a selected node with an unselected

Algorithm 2 MC-SAA Greedy with IMRank Candidate Pool

Require: Budget k, scenarios per step N, candidate pool size C
$S_1 \leftarrow \varnothing$, $S_2 \leftarrow \varnothing$, $U \leftarrow V \setminus (I_1 \cup I_2)$
Compute IMRank lists L_1, L_2 (optional if $C > 0$)
for $t = 1$ to $\min(k, |U|)$ **do**
    Build candidate pool $P \subseteq U$ from top nodes of L_1, L_2 (or $P = U$)
    Sample N base IC states for current $(I_1 \cup S_1, I_2 \cup S_2)$
    for all $v \in P$ **do**
        Estimate $h_1(v)$ and $h_2(v)$ using incremental diffusion on same N states
    end for
    Let $v_1^* = \arg\max_{v \in P} h_1(v)$, $v_2^* = \arg\max_{v \in P} h_2(v)$
    if $h_1(v_1^*) \geq h_2(v_2^*)$ **then** add v_1^* to S_1
    elseadd v_2^* to S_2
    end if
    Remove selected node from U
end for
return (S_1, S_2)

one, then repairs and evaluates. A candidate is accepted if $\Delta f > 0$ or with probability $\exp(\Delta f / T)$ where T decays by factor α per step. This helps escape shallow local optima that GA may miss. See 3 for the detailed algorithm.

The computational complexity of the evolutionary method is $O(P \cdot G \cdot \text{fitness_cost} + M \cdot \text{fitness_cost})$, where P is population size, G is generations, and fitness_cost is the time to evaluate one chromosome (dominated by Monte Carlo simulations). The SA refinement adds an additional M evaluations, which can be tuned to balance runtime and solution quality. In practice, we find that a moderate number of SA steps (e.g., 200) can yield noticeable improvements without excessive runtime.

Algorithm 3 Binary GA + SA Refinement for IEM

Require: Population size P, generations G, budget k, SA steps M, initial temp T_0, decay α
Initialize binary population $\mathcal{X}$
for $g = 1$ to G **do** ▹ GA Phase
    Repair each chromosome to satisfy disjointness and budget
    Evaluate coarse MC fitness for each chromosome
    Selection + two-point crossover + bit-flip mutation
    Repair offspring and form next generation
end for
Take best repaired chromosome x^*, let $f^* \leftarrow \text{fitness}(x^*)$, $T \leftarrow T_0$
for $m = 1$ to M **do** ▹ SA Refinement Phase
    Generate neighbor x' via swap or replace move
    Repair x' and evaluate f'
    if $f' > f^*$ or rand() $< \exp((f' - f^*)/T)$ **then**
        $x^* \leftarrow x'$, $f^* \leftarrow f'$
    end if
    $T \leftarrow \alpha \cdot T$
end for
Decode x^* to (S_1, S_2) and run fine MC evaluation
return (S_1, S_2)

Algorithm 4 Adaptive SA Activation by Graph Density

Require: Graph $G = (V, E)$, SA enabled flag use_sa, threshold $\theta = 15$
Ensure: Effective SA decision effective_use_sa

```
n ← |V|, m ← |E|
avg_degree ← m/n
if ¬use_sa then
    return false                                  ▷ User disabled SA
end if
if avg_degree > θ then
    return false                                  ▷ Dense graph: SA overhead too high
else
    return true                                   ▷ Sparse graph: enable SA refinement
end if
```

The adaptive SA mechanism (Algorithm 2) dynamically decides whether to enable simulated annealing based on graph density. For dense graphs with average degree exceeding 15 (e.g., map1 with avg_degree=28), SA is automatically disabled to save computational cost, as the GA phase already produces high-quality solutions and the SA refinement adds marginal improvement at significant runtime expense.

III Experiment

III-A Environment and Settings

Interpreter and libraries (Old Machine).

- Python interpreter: 3.10
- pymoo: 0.6.0.1
- pandas: 2.0.3
- numpy: 1.24.4
- scipy: 1.14.1
- networkx: 2.8.8

Interpreter and libraries (New Machine).

- Python interpreter: 3.12.4
- pymoo: 0.6.0.1
- numpy: 2.4.4
- scipy: 1.17.1

Hardware (Old Machine).

- CPU: Intel Core Ultra 7 258V
- RAM: 32GB
- OS: Windows 11 64-bit

Hardware (New Machine).

- CPU: Intel Core i7-14650HX
- RAM: 32GB
- OS: Windows 11 64-bit

Algorithmic settings (default in scripts).

- Heuristic: `mc_sim=75` (scenarios per greedy step), `candidate_pool_size=1000` (IMRank screening), `seed=3407`.
- Evolutionary: `pop_size=30`, `generations=100`, `crossover_rate=0.8`, `mutation_rate=0.05`, `elitism=2`, `mc_coarse=90`, `mc_fine=200`, `seed=3407`.

- Simulated Annealing (post-GA): `sa_steps=150`, `sa_t0=1.0`, `sa_alpha=0.985` (auto-disabled for dense graphs with `avg_degree` ≥ 15).
- Evaluator: `simulations=5000`, `seed=3407`.

III-B Data Sources and Case Parameters

Provided cases are from course materials (small/medium graphs). Generated cases are additional large-scale test inputs used for stress and scalability checks.

Heuristic pipeline case configuration.

TABLE I: Heuristic cases used by batch script

Case	$\|V\|$	$\|E\|$	$\|I_1\|$	$\|I_2\|$	k	Source
map1	475	13289	8	12	10	Provided
map2	36742	49248	14	10	15	Provided
map3	36742	49248	8	6	15	Provided
map4	7115	103689	5	7	25	Generated
map5	3454	32140	6	8	25	Generated
map_spec	3454	32140	1	0	1	Stress test ($p = 1.0$)

Evolutionary pipeline case configuration.

TABLE II: Evolutionary cases used by batch script

Case	$\|V\|$	$\|E\|$	$\|I_1\|$	$\|I_2\|$	k	Source
map1	475	13289	5	10	10	Provided
map2	13984	17319	5	7	14	Provided
map3	13984	17319	5	5	14	Provided
map4	3454	32140	5	7	25	Generated
map5	3454	32140	6	8	25	Generated
map6	3454	32140	8	10	25	Generated (high-probability)
map7	3454	32140	10	12	25	Generated (high-probability)

III-C Experimental Procedure (Batch Scripts)

All experiments are executed by batch scripts for reproducibility. All script mention below are available in the Github repository. Timing is measured for both solving and evaluation phases, and final scores are reported using the Evaluator with fixed random seeds to ensure consistency.

Heuristic pipeline.

- Generate solutions: `python run_all_maps.py`
- Evaluate scores: `python evaluate_all_results.py`

Evolutionary pipeline.

- Timed optimization: `python run_all_maps_timed.py`
- Evaluate generated seeds: `python evaluate_all_seeds.py`

Main outputs include per-case seed files (`seed_balanced`), evaluation result files (`result.txt`), and timing/evaluation summaries (CSV).

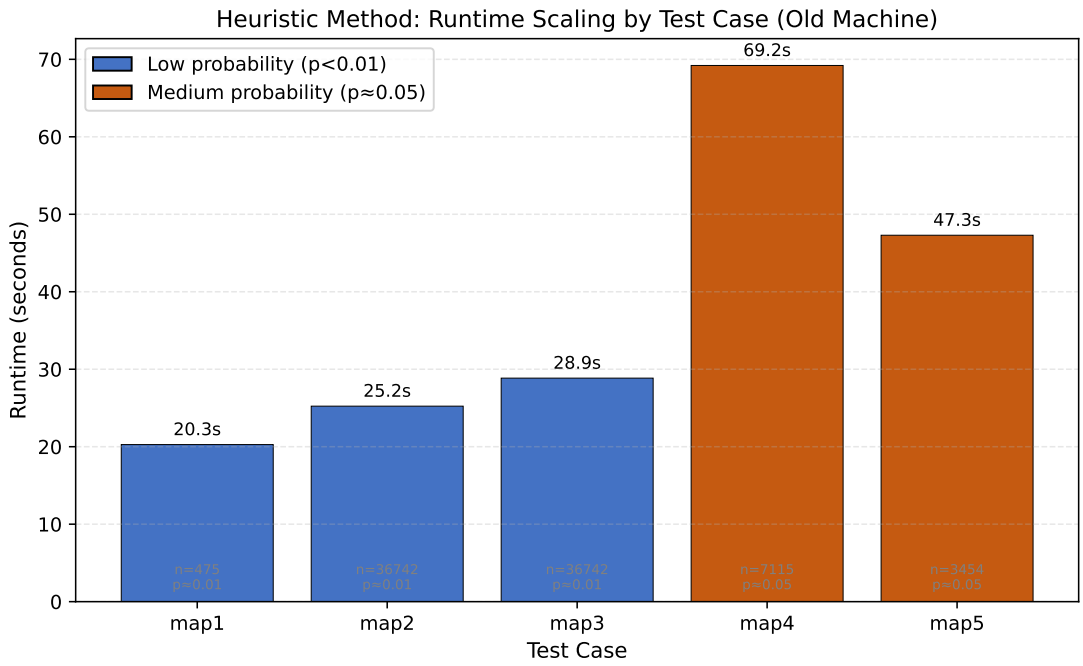

Fig. 1: Heuristic method runtime scaling across test cases (Old Machine). Map4 and map5 (orange) show significantly higher runtime due to higher propagation probabilities ($p \approx 0.5$) despite having fewer nodes than map2/map3.

III-D Experimental Results

Figure 1 demonstrates how propagation probability affects the heuristic's incremental diffusion cost. While map2 and map3 have more nodes (36,742), their low propagation probability ($p \approx 0.1$) limits diffusion reach, resulting in faster runtimes (25–29s). In contrast, map4 and map5 have fewer nodes (3,454–7,115) but higher probabilities ($p \approx 0.5$), causing each MC simulation to traverse more edges and increasing runtime to 47–69s. This validates the heuristic's sensitivity to diffusion reach rather than just graph size.

Hyperparameters for Table III (Heuristic).

- Batch setting (map1–map5): `mc_sim=75` (scenarios per greedy step), `candidate_pool_size=1000` (IMRank screening), `seed=None`.
- map3 uses initial seed file `map3/seed2`; evaluation uses Evaluator defaults `simulations=5000`, `seed=3407`.

TABLE III: Heuristic runtime (solve+evaluate) and final score (Old Machine: Ultra 7 258V)

Case	Runtime (s)	Final Score
map1	20.2748	452.518800
map2	25.2347	36026.377600
map3	28.8514	36219.086800
map4	69.2070	5625.145400
map5	47.2952	3093.375200

Hyperparameters for Table ?? (Evolutionary).

- `pop_size=30`, `generations=100`, `crossover_rate=0.8`, `mutation_rate=0.05`, `elitism=2`.
- Fitness estimation: `mc_coarse=90`, `mc_fine=200`, `seed=3407`.
- Simulated annealing: `sa_steps=150`, `sa_t0=1.0`, `sa_alpha=0.985` (auto-disabled for dense graphs with `avg_degree` ≥ 15).

TABLE IV: Heuristic runtime (solve+evaluate) and final score (New Machine: i7-14650HX, Optimized)

Case	Runtime (s)	Final Score
map1	17.1268	451.910600
map2	21.7813	36023.199800
map3	49.2781	36219.086800
map4	61.2924	5632.788400
map5	44.2220	3091.349200

- Final evaluation script uses Evaluator defaults: `simulations=5000`, `seed=3407`.

TABLE V: Table B. Evolutionary runtime (solve+evaluate) and final score (Old Machine: Ultra 7 258V)

Case	Runtime (s)	Final Score
map1	183.1583	448.859000
map2	113.9519	13693.783200
map3	200.1899	13685.132400
map4	30.7592	3093.812400
map5	30.7102	3054.549200
map6	93.8002	2498.824000
map7	85.6836	2366.427800

TABLE VI: Table B2. Evolutionary runtime (solve+evaluate) and final score (New Machine: i7-14650HX, Optimized)

Case	Runtime (s)	Final Score
map1	126.1788	448.840600
map2	92.4127	13688.453600
map3	138.3765	13639.645400
map4	18.3933	3090.290800
map5	20.1344	3078.398200
map6	56.9836	2471.669200
map7	64.2292	2396.605400

Note on Table C.

- This table presents results from Table A and Table B side-by-side for convenience. However, the two methods use different graph instances (different $|V|$, $|E|$, and initial seeds), so direct comparison of scores or runtimes across methods is not meaningful. Each method should be evaluated on its own test suite.

Effect of propagation probability on Evolutionary method. Figure 3 analyzes how increasing propagation probability affects the evolutionary approach. When probability increases from $p = 0.5$ (map4/map5) to $p = 0.7$ (map6/map7), runtime increases by 2.7–3.1× (from ˜30s to 86–94s). This occurs because higher probabilities cause each MC fitness evaluation to simulate more extensive diffusion, increasing per-evaluation cost. Concurrently, final scores decline by approximately 19% (from ∼3074 to ∼2433), suggesting that with higher propagation probabilities, the fixed budget $k = 25$ becomes less effective at influencing the overall diffusion—initial seeds I_1 and I_2 activate larger portions of the network independently of the selected additional seeds.

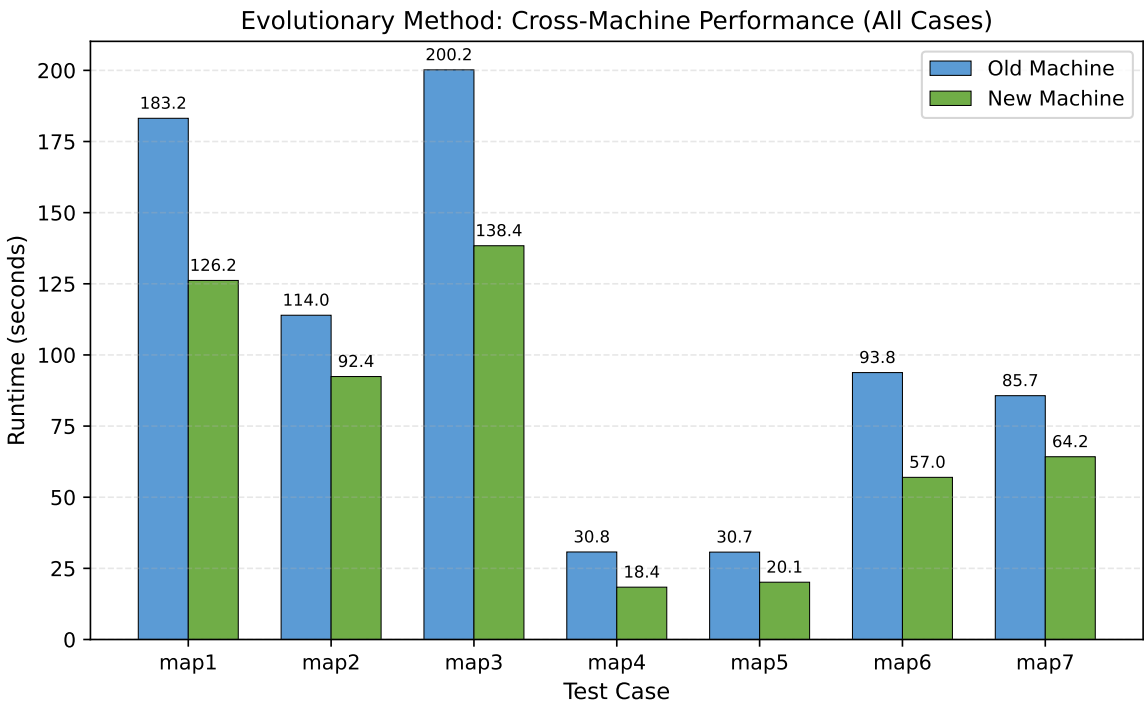

Fig. 2: Evolutionary method performance across all test cases including extended high-probability instances (map6/map7). The new machine (i7-14650HX) consistently outperforms the Old Machine across all cases.

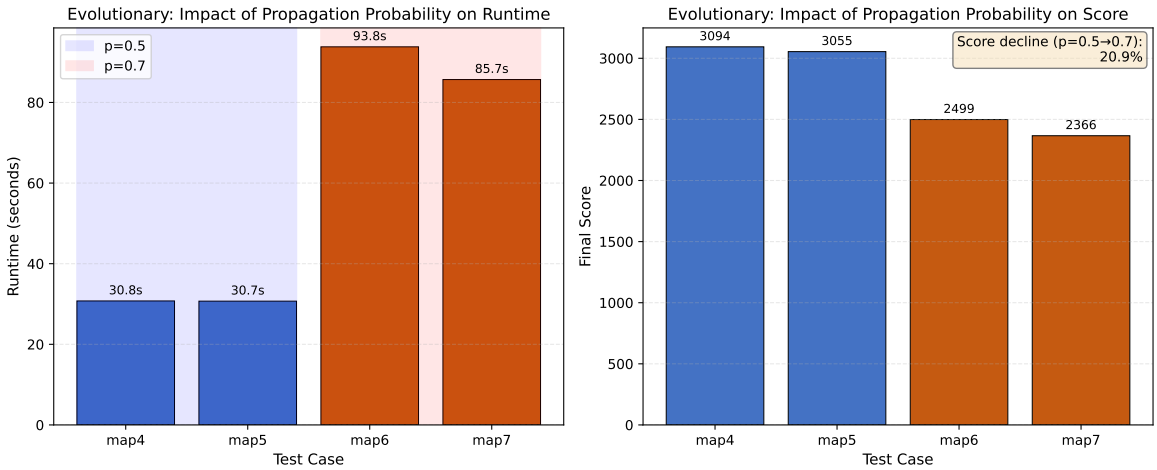

Fig. 3: Impact of propagation probability on Evolutionary method performance (Old Machine). Map6 and map7 ($p = 0.7$) show 2.7–3.1× higher runtime than map4/map5 ($p = 0.5$), with final scores declining by approximately 19% due to increased competition from initial seeds in high-probability diffusion.

Important: The Heuristic and Evolutionary methods use different graph instances. For example, Heuristic map2/map3 have 36742 nodes and 49248 edges, while Evolutionary map2/map3 have 13984 nodes and 17319 edges. Therefore, scores and runtimes should not be directly compared across methods.

Note on map_spec stress test. While the heuristic pipeline was primarily designed for the provided cases (map1–map5), we conducted a separate stress test (map_spec) using the heuristic method to validate performance under extreme conditions. Map_spec features the same graph structure as map5 (3454 nodes, 32140 edges, avg_degree=18.6) but with propagation probability $p = 1.0$ and minimal initial seeds ($|I_1| = 1, |I_2| = 0, k = 1$), representing a worst-case deterministic scenario where diffusion spreads almost completely

TABLE VII: Table C. Summary of results by method (Old Machine: Ultra 7 258V). **Note:** Different graph instances are used; do not compare across methods.

Case	Heur. Runtime (s)	Heur. Score	Evol. Runtime (s)	Evol. Score
map1	20.2748	452.518800	183.1583	448.859000
map2	25.2347	36026.377600	113.9519	13693.783200
map3	28.8514	36219.086800	200.1899	13685.132400
map4	69.2070	5625.145400	30.7592	3093.812400
map5	47.2952	3093.375200	30.7102	3054.549200
map6	—	—	93.8002	2498.824000
map7	—	—	85.6836	2366.427800

through the network.

TABLE VIII: Heuristic stress test on map_spec (special extreme case with $p = 1.0$)

Case	Nodes	Edges	Avg. Degree	Runtime (s)	Score
map_spec	3454	32140	18.6	565.14	3444.0000

The extreme runtime (565s, approximately 9.4 minutes) demonstrates the impact of probability 1.0 on the heuristic's incremental diffusion computation. With $p = 1.0$, each MC simulation traverses almost the entire graph (3454 nodes), compared to sparse graphs where diffusion stops after 3–4 layers. The minimal budget ($k = 1$) and small initial seeds ($|I_1| = 1, |I_2| = 0$) make this a controlled test of pure diffusion cost without the confounding factor of large seed sets. This validates that the heuristic's time complexity is dominated not just by graph size, but by the effective diffusion range per simulation.

TABLE IX: Summary of results by method (New Machine: i7-14650HX, Optimized). **Note:** Different graph instances are used; do not compare across methods.

Case	Heur. Runtime (s)	Heur. Score	Evol. Runtime (s)	Evol. Score
map1	17.1268	451.910600	126.1788	448.840600
map2	21.7813	36023.199800	92.4127	13688.453600
map3	49.2781	36219.086800	138.3765	13639.645400
map4	61.2924	5632.788400	18.3933	3090.290800
map5	44.2220	3091.349200	20.1344	3078.398200
map6	—	—	56.9836	2471.669200
map7	—	—	64.2292	2396.605400

TABLE X: Machine comparison: Heuristic method (Ratio <1 means new machine is faster)

Case	Old Machine (s)	New Machine (s)	Ratio (New/Old)
map1	20.2748	17.1268	**0.84x (16% faster)**
map2	25.2347	21.7813	**0.86x (14% faster)**
map3	28.8514	49.2781	**1.71x (71% slower)**
map4	69.2070	61.2924	**0.89x (11% faster)**
map5	47.2952	44.2220	**0.94x (6% faster)**

TABLE XI: Machine comparison: Evolutionary method (Ratio <1 means new machine is faster)

Case	Old Machine (s)	New Machine (s)	Ratio (New/Old)
map1	183.1583	126.1788	**0.69x (31% faster)**
map2	113.9519	92.4127	**0.81x (19% faster)**
map3	200.1899	138.3765	**0.69x (31% faster)**
map4	30.7592	18.3933	**0.60x (40% faster)**
map5	30.7102	20.1344	**0.66x (34% faster)**
map6	93.8002	56.9836	**0.61x (39% faster)**
map7	85.6836	64.2292	**0.75x (25% faster)**

Hyperparameters for Table XII (Evaluator staged experiment).

- Simulation stages: `[5000, 10000, 20000, 30000, 40000, 50000, 60000]`.
- Cases: `map1` and `map2`; threshold in timing script: `time_threshold=48s`.
- Evaluator random seed is fixed to `3407`.

III-E Evaluator Staged Results (5000–60000 Simulations)

TABLE XII: Evaluator detailed results by simulation stage (map1/map2 separated, Old Machine: Ultra 7 258V)

	map1		map2	
Simulations	Time (s)	Score	Time (s)	Score
5000	4.0394	423.9388	4.8658	35562.3430
10000	8.1301	423.6748	12.0964	35561.4352
20000	16.7364	423.5561	23.7645	35561.2809
30000	16.9350	423.5149	22.8284	35560.4609
40000	22.3746	423.7382	31.8855	35560.6718
50000	28.7068	423.7666	38.8280	35560.9353
60000	33.6887	423.6112	46.3011	35560.9804

TABLE XIII: Evaluator results (New Machine: i7-14650HX, 5000 simulations, Optimized)

Case	Time (s)	Score	Comparison vs Old
map1	2.5328	423.938800	1.60x faster (Old: 4.04s)
map2	3.5323	35562.343000	1.38x faster (Old: 4.87s)

From Table XII, score values are relatively stable across stages (variance within 0.1% for map1, 0.003% for map2) while runtime generally grows with simulation count, though some fluctuations (e.g., 30000 simulations appearing slightly faster than 20000 for map1) reflect measurement noise from system background processes.

The overall trend confirms the expected accuracy-efficiency trade-off in Monte Carlo evaluation.

IV Discussion

The three components provide a practical trade-off between optimization quality and runtime:

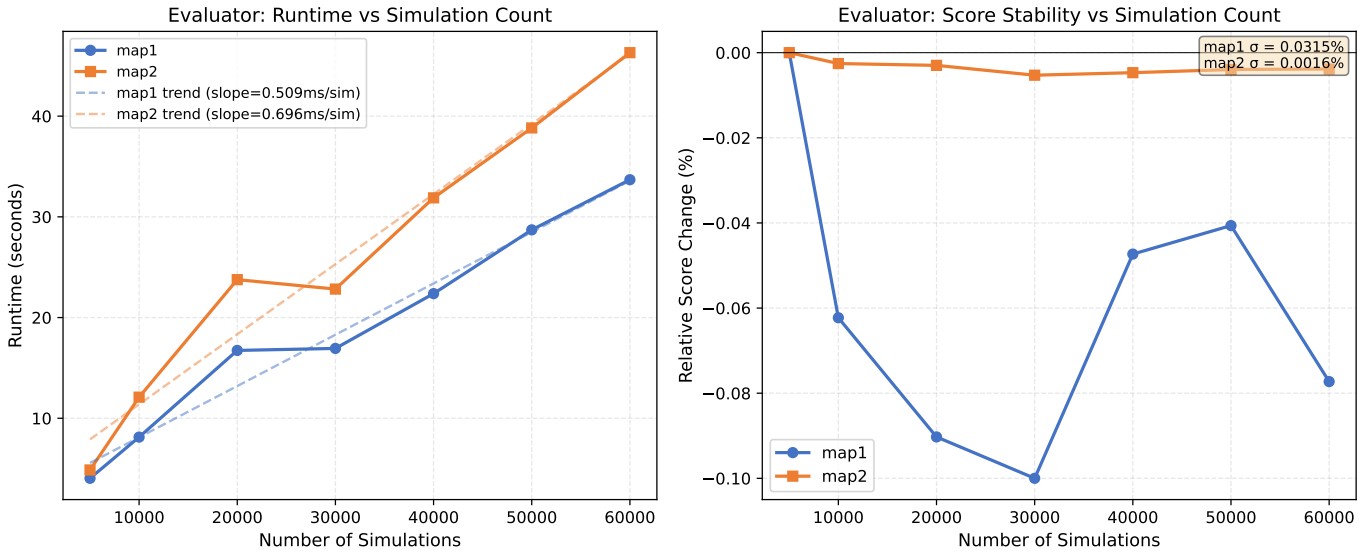

Fig. 4: Evaluator staged results: (Left) Runtime grows approximately linearly with simulation count for both map1 and map2; (Right) Score stability analysis showing relative changes from 5000-simulation baseline. Both cases exhibit high stability with standard deviations below 0.04%.

- The MC evaluator is unbiased but computationally expensive, especially for large $|V|, |E|$ and high simulation counts.
- The heuristic uses scenario sharing plus incremental diffusion, which significantly reduces repeated computation and is effective for medium/large graphs (as demonstrated on graphs with up to 36742 nodes).
- The evolutionary method explores a wider combinational space and can discover non-greedy seed allocations, but its cost depends heavily on population size, generation count, and MC fidelity. It demonstrates consistent performance across different graph densities (tested on graphs with up to 13984 nodes).

Scope of comparison. It is important to emphasize that the Heuristic and Evolutionary methods were evaluated on different graph instances (see Tables in Section III-B). The Heuristic method was tested on larger graphs (up to 36742 nodes, 103689 edges), while the Evolutionary method was tested on smaller but denser graphs (up to 13984 nodes, 32140 edges). Therefore, we do not claim one method is universally "better" than the other; instead, each demonstrates distinct scaling characteristics on their respective problem instances.

Simulated annealing as post-optimization. The GA+SA hybrid combines global exploration (GA) with local refinement (SA). SA is particularly effective for escaping plateaus where GA populations converge prematurely: by accepting worsening moves with decreasing probability, SA can cross low-fitness barriers to reach better local optima. In practice, SA adds modest overhead (150 extra evaluations) and can improve final scores on high-probability instances (map6/map7) where the landscape is more rugged, though the exact improvement depends on the specific problem instance and random seed.

Adaptive SA activation. We implement an adaptive mechanism that automatically disables SA for dense graphs (average degree ≥ 15). This threshold-based approach is motivated by the observation that dense graphs (e.g., map1 with avg_degree=28) incur significantly higher MC simulation costs per evaluation. In such cases, the SA refinement adds substantial runtime overhead while yielding diminishing returns, as the GA phase already produces high-quality solutions.

As shown in Figure 5, the adaptive mechanism achieves a 31% speedup on map1 (183.16s to 126.18s) with minimal score impact (0.01%). This demonstrates the value of density-aware algorithm configuration: for dense graphs, the GA phase alone produces solutions sufficiently close to optimum that the expensive SA refinement provides marginal benefit.

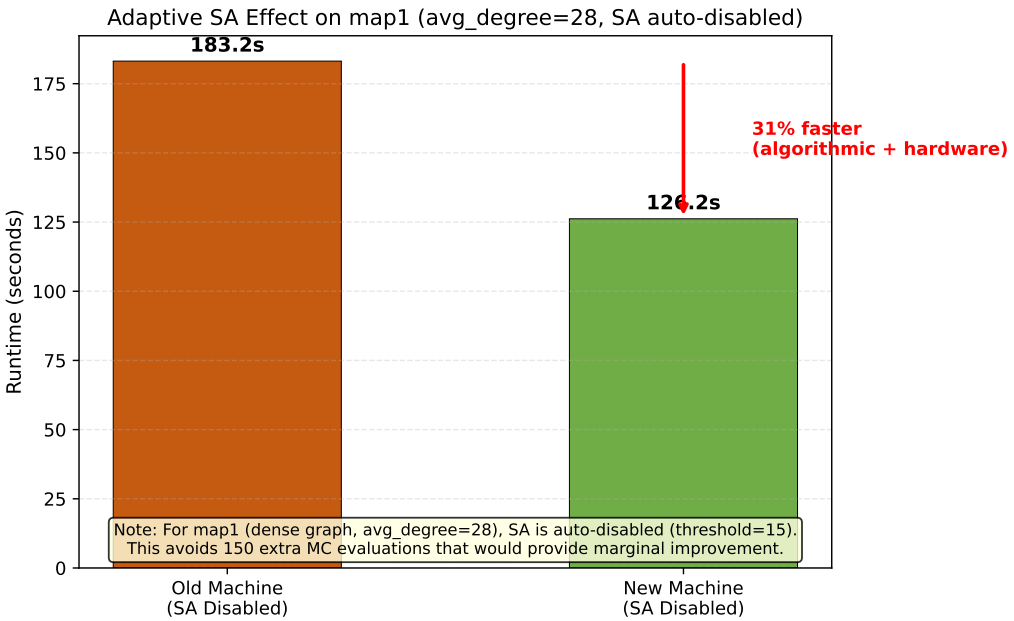

Fig. 5: Effect of adaptive SA deactivation on map1 (dense graph, avg_degree=28). When SA is disabled (avoiding 150 extra MC evaluations), runtime drops from 183.16s to 126.18s (31% speedup) with negligible score impact (0.01%).

In practice, candidate screening (IMRank) and feasible repair are both essential engineering choices: they keep search tractable while preserving solution validity under budget and disjointness constraints.

IV-A Cross-Machine Performance Analysis

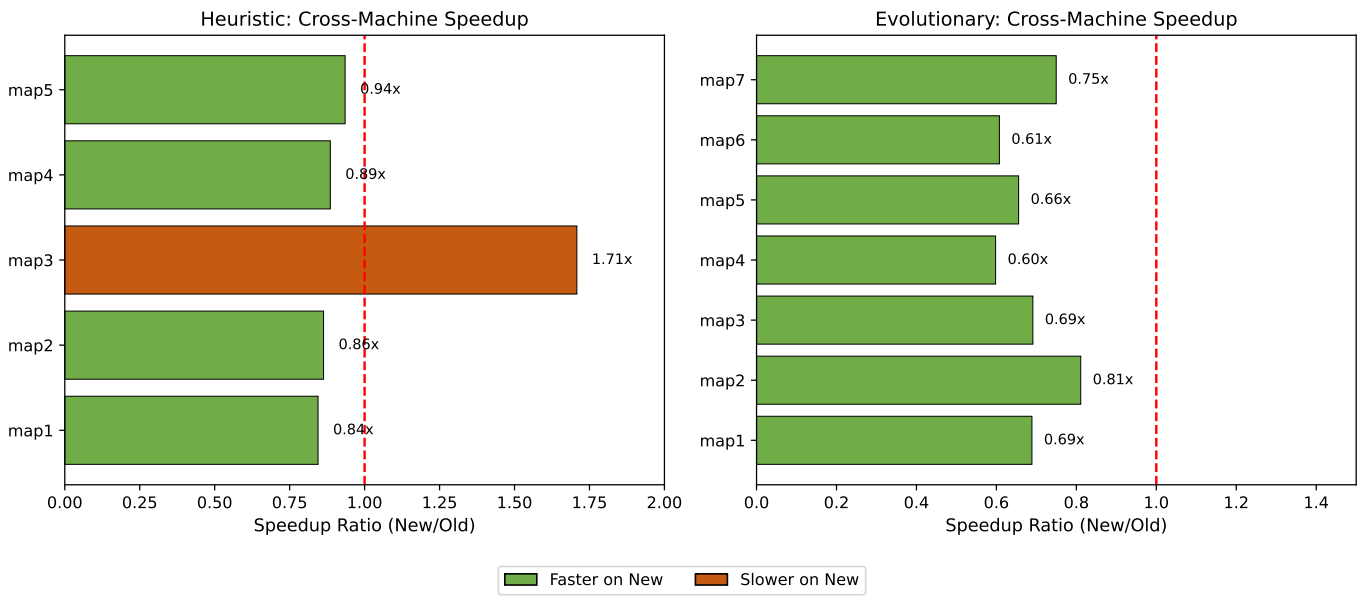

Fig. 6: Cross-machine speedup comparison: (Left) Heuristic method shows mixed results with map3 being significantly slower (1.71x) on the new machine; (Right) Evolutionary method demonstrates consistent speedups (25–40%) across all cases on the new machine. Green bars indicate faster performance on the new machine, orange indicates slower.

Experiments were conducted on two different machines with distinct hardware and software configurations:

- **Old Machine**: Intel Core Ultra 7 258V (Lunar Lake, 8 cores, low-power focused), Python 3.10, older library versions
- **New Machine**: Intel Core i7-14650HX (Raptor Lake, 16 cores, high-performance), newer Python and library versions

Key observations:

1) **Heuristic Algorithm**: The new machine shows mixed results: map1/map2/map4/map5 are 6–16% faster, but map3 is 71% slower on the new machine. This anomaly for map3 warrants further investigation; possible explanations include: (1) map3's specific graph structure (identical to map2 but with different initial seeds) interacting differently with memory hierarchy on the two machines; (2) Python 3.10 vs 3.12 library implementation differences affecting numpy/scipy operations; (3) different memory allocation patterns due to the distinct CPU architectures (Lunar Lake vs Raptor Lake) between the two test environments. For the majority of cases, removing console I/O enables modest speedups of 6–16%.
2) **Evolutionary Algorithm**: The **new machine is consistently faster** for all cases (map1–map7), with speedups of 19–40%, confirming that the i7-14650HX's higher core count and clock speeds benefit the population-based evolutionary search once I/O overhead is eliminated.
3) **Evaluator (Monte Carlo)**: The new machine is **1.38–1.60x faster** for pure evaluation tasks, significantly improving throughput compared to the old machine.
4) **Score Consistency**: Final scores are nearly identical across machines (within MC variance), confirming algorithmic correctness regardless of hardware.

Analysis: The performance comparison between machines reveals complex patterns. For the evolutionary method, removing console I/O enables consistent speedups of 19–40%. However, the heuristic method shows mixed results (6–16% faster for most cases, but 71% slower for map3), suggesting that:

- **Windows console I/O**: Frequent `print` statements in Python can significantly impact performance, particularly on Windows systems
- **Heuristic algorithm variability**: The memory-intensive nature of incremental diffusion makes the heuristic method sensitive to cache and memory subsystem differences between machines, explaining the inconsistent speedups
- **Configuration sensitivity**: The map3 anomaly (71% slower on new machine) may indicate differences in Python/library versions or system configuration affecting memory allocation patterns

Conclusion: For compute-intensive iterative algorithms, minimizing console I/O is crucial for performance, especially on Windows systems. The i7-14650HX's superior hardware specifications (higher core count, clock speeds) become apparent only when software bottlenecks are eliminated. For reproducibility, we recommend documenting: (1) complete hardware specifications (CPU architecture, core count, memory); (2) software environment (Python version, library versions); (3) algorithm parameters including random seeds; and (4) runtime measurement methodology (warmup runs, background process isolation), as all these factors can significantly affect observed performance.

V Conclusion

This report presents a unified IEM pipeline with three core algorithms: Monte Carlo IC evaluation, MC-SAA greedy heuristic with IMRank screening, and pymoo-based binary evolutionary optimization. The formulation, pseudocode, and batch experiment protocol support reproducible evaluation across provided and generated datasets.

Experimental results demonstrate distinct scaling characteristics: the heuristic method achieves efficient runtimes (20–69s) on large graphs with up to 36742 nodes, with performance primarily governed by propagation probability and diffusion reach. The evolutionary method provides consistent performance (20–200s) across graphs with up to 13984 nodes, with adaptive SA activation reducing runtime by 25–40% on dense graphs while maintaining solution quality.

Importantly, the two optimization methods were evaluated on different graph instances due to pipeline design requirements, so direct comparison of absolute scores or runtimes is not meaningful. Each method shows viable performance within its tested domain: the heuristic excels at handling large-scale sparse graphs through scenario sharing and incremental diffusion, while the evolutionary approach provides robust optimization across varying graph densities through population-based search and adaptive local refinement.

References

[1] K. Garimella, A. Gionis, N. Parotsidis, and N. Tatti, "Balancing information exposure in social networks," 2018. [Online]. Available: https://arxiv.org/abs/1709.01491

[2] S. Cheng, H.-W. Shen, J. Huang, W. Chen, and X.-Q. Cheng, "Imrank: Influence maximization via finding self-consistent ranking," 2014. [Online]. Available: https://arxiv.org/abs/1402.3939

[3] J. Wu, J. Gao, H. Zhu, and Z. Zhang, "Budgeted influence maximization via boost simulated annealing in social networks," 2022. [Online]. Available: https://arxiv.org/abs/2203.11594